项目 4 底层运动控制

1、 预备知识

1.1、 关节序号与关节限位

四足机器人像动物一样，身体（Trunk）和四肢（Leg）都是左右对称的。四条腿，按照前后分成两组，这两组除了前后不同，坐标系和关节活动范围等都相同的。

腿和关节的序号
Leg 0： FR，右前腿
Leg 1： FL，左前腿
Leg 2： RR，右后腿
Leg 3： RL，左后腿
Joint 0： Hip，机身关节
Joint 1： Thigh，大腿关节
Joint 2： Calf，小腿关节
e.g.FR_thigth：右前腿大腿关节

各关节限位
机身关节：-48°~48°
大腿关节：竖直方向为 0°，前进方向为正，后腿-260°~30°，前腿-200°~90°
小腿关节： -156°~-48°

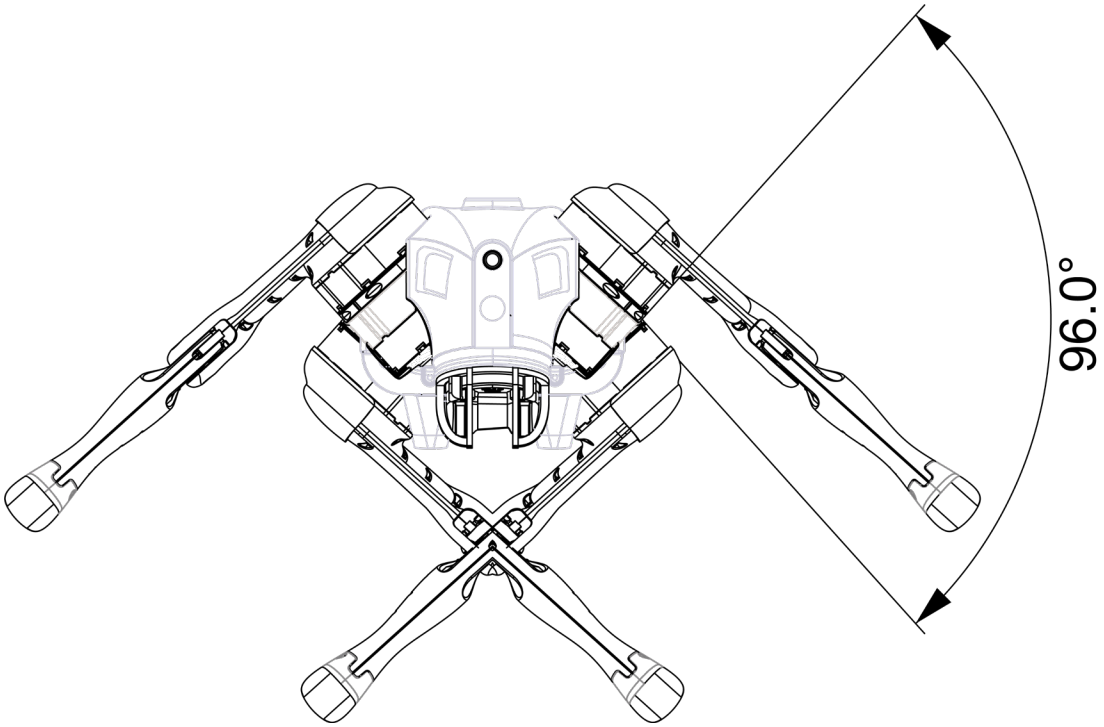


图 1-1 机身关节限位

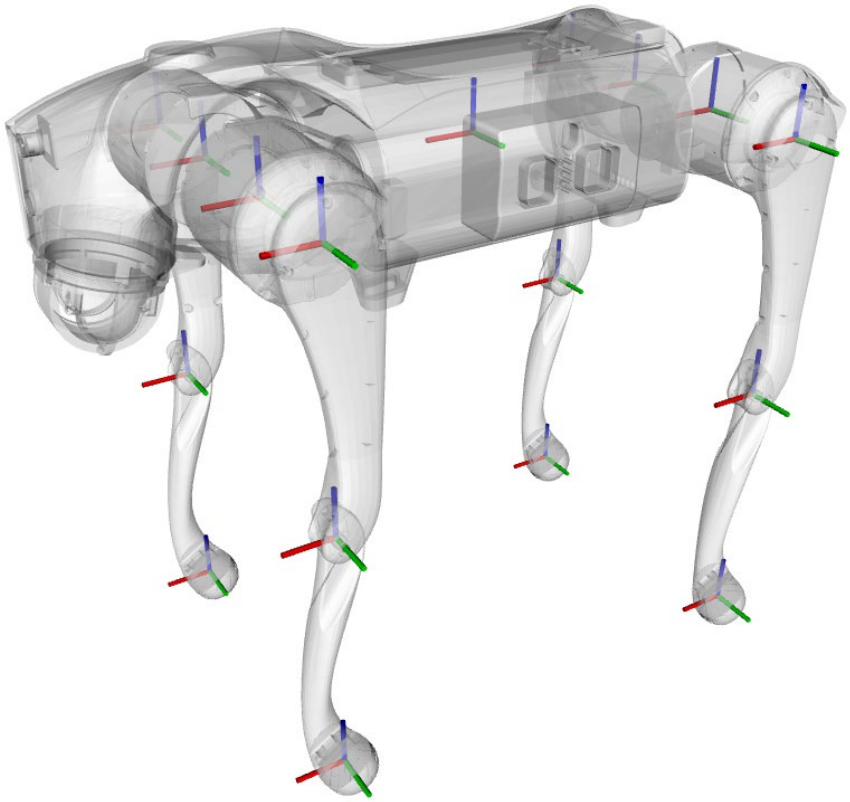


图 1-2 坐标系、关节旋转轴与关节零点

机身关节旋转轴为 x 轴，大腿关节和小腿关节的旋转轴为 y 轴，旋转正方向符合右手定则。

当各个关节均为零度时，各坐标系如上图。红色为 x 轴，绿色为 y 轴，蓝色为 z 轴。小腿关节由于限位，实际到不了这个位置，这里仅为了指明零点位置。可以看出，各个关节坐标系的初始姿态都是一致的，只是位置和旋转轴不同。

1.2、 主运控服务与底层运动控制

我们在运行底层运动控制程序时，需要先关闭主运控服务(sport_mode)，这是因为底层控制例程也相当于一个运控服务，它们均会发送控制指令给 Go2，如果多个运控同时存在，则 Go2 机器人会同时接收两个或多个控制指令而产生混乱，造成机器狗失控。故在运行底层控制例程序前，需要确保对应的服务处于关闭状态。

1.3、 底层服务接口

底层运动控制基于底层服务接口，底层服务接口用于获取电机，电池，遥控器，IMU 数据并订阅于 **rt/lowstate 话题**，发布控制命令基于 **rt/lowcmd** 话题并控制电机、电池。

1.3.1、 底层控制指令

用户可通过发布 DDS 话题 “rt/lowcmd” 来发送电机、电池、自动充电、电机电源开关的控制指令，数据格式如以下的 LowCmd_.idl 所示。

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
// with input from unitree_go:msg/LowCmd.idl
// generated code does not contain a copyright notice
#include "BmsCmd_.idl"
#include "MotorCmd_.idl"
#ifndef __unitree_go__msg__low_cmd__idl__
#define __unitree_go__msg__low_cmd__idl__

module unitree_go {

module msg {

module dds_ {
```

```
struct LowCmd_ {
octet head[2];                    //帧头，数据校验用（0xFE,0xEF）。

octet level_flag;                 //保留，目前不用。
octet frame_reserve;              //保留，目前不用。
unsigned long sn[2];              //保留，目前不用。
unsigned long version[2];   //保留，目前不用。
unsigned short bandwidth;   //保留，目前不用。

// FR_0 -> 0 , FR_1 -> 1   , FR_2 -> 2    电机控制顺序，目前只用12 电机，后面保留。
// FL_0 -> 3 , FL_1 -> 4   , FL_2 -> 5
// RR_0 -> 6 , RR_1 -> 7   , RR_2 -> 8
// RL_0 -> 9 , RL_1 -> 10 , RL_2 -> 11
unitree_go::msg::dds_::MotorCmd_ motor_cmd[20];    //电机控制命令数据
unitree_go::msg::dds_::BmsCmd_  bms_cmd;                  //电池
```

控制命令数据

octet wireless_remote[40]; //保留，目前不用。

octet led[12]; //已经改为内部控制，目前不用。

octet fan[2]; //已经改为内部控制，目前不用。

// &0xFE 自动充电打开 , |0x01 自动充电关闭

// &0xFD 12 个电机的电源开关打开 , |0x02 12 个电机的电源开关关闭

octet gpio;

unsigned long reserve; //保留位

unsigned long crc; //数据 CRC 校验用,为 32crc 校验用。

};

}; // module dds_

```
    };  // module msg


    };  // module unitree_go



    #endif  // __unitree_go__msg__low_cmd__idl__
```

电机控制命令：机身共 12 个电机，控制顺序可以按照如下进行，详情见以下的 MotorCmd_.idl 。

```
#ifndef __unitree_go__msg__motor_cmd__idl__

#define __unitree_go__msg__motor_cmd__idl__



module unitree_go {

module msg {

module dds_ {
```

```
    struct MotorCmd_ {
    octet mode;  //电机控制模式（Foc 模式（工作模式）-> 0x01 ， stop
模式（待机模式）-> 0x00。）
    float q;       //关节目标位置
    float dq;      //关节目标速度
    float tau;     //关节目标力矩
    float kp;      //关节刚度系数
    float kd;      //关节阻尼系数
    unsigned long reserve[3];     //保留位


    };


    };   // module dds_
```

```
};  // module msg


};  // module unitree_go



#endif  // __unitree_go__msg__motor_cmd__idl__
```

电池控制命令：关闭机身电池，详情见以下的 BmsCmd_.idl 。

```
#ifndef __unitree_go__msg__motor_cmd__idl__

#define __unitree_go__msg__motor_cmd__idl__



module unitree_go {


module msg {


module dds_ {
```

```
    struct MotorCmd_ {
    octet mode;  //电机控制模式（Foc 模式（工作模式）-> 0x01 ， stop
模式（待机模式）-> 0x00。）
    float q;       //关节目标位置
    float dq;      //关节目标速度
    float tau;     //关节目标力矩
    float kp;      //关节刚度系数
    float kd;      //关节阻尼系数
    unsigned long reserve[3];     //保留位



    };



    };  // module dds_

    };  // module msg
```

```
};  // module unitree_go



#endif  // __unitree_go__msg__motor_cmd__idl__
```

自动充电命令：控制自动充电的命令，&0xFE->自动充电打开，|0x01->自动充电关闭。

电机电源控制命令：控制电机电源的命令，&0xFD->12 个电机的电源开关打开，|0x02->12 个电机的电源开关关闭。

1.3.2、 底层数据接收

用户可通过订阅 DDS 话题“**rt/lowState**” 来获取电机、电池、IMU、遥控器数据，数据格式见 以下的 **LowState_.idl** 。

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em

// with input from unitree_go:msg/LowState.idl

// generated code does not contain a copyright notice
```

```
#include "BmsState_.idl"

#include "IMUState_.idl"

#include "MotorState_.idl"


#ifndef __unitree_go__msg__low_state__idl__

#define __unitree_go__msg__low_state__idl__




module unitree_go {


module msg {


module dds_ {



struct LowState_ {

octet head[2];                    //帧头，数据校验用（0xFE,0xEF）。
```

```
    octet level_flag;                   //沿用的，但是目前不用。
    octet frame_reserve;                //沿用的，但是目前不用。
    unsigned long sn[2];                //已经改为文件存储形式，目前没用。
    unsigned long version[2];           //沿用的，但是目前不用。
    unsigned short bandwidth;           //沿用的，但是目前不用。。

    unitree_go::msg::dds_::IMUState_ imu_state;                 //IMU 数据
信息。

    // FR_0 -> 0 , FR_1 -> 1   , FR_2 -> 2    电机顺序，目前只用 12 电机，
后面保留。
    // FL_0 -> 3 , FL_1 -> 4   , FL_2 -> 5
    // RR_0 -> 6 , RR_1 -> 7   , RR_2 -> 8
    // RL_0 -> 9 , RL_1 -> 10 , RL_2 -> 11
    unitree_go::msg::dds_::MotorState_ motor_state[20];     //电机总数
据。
    unitree_go::msg::dds_::BmsState_ bms_state;                 //电池总数
据。
```

```
    short foot_force[4];            //足端力（范围 0-4095），可按照实际数值显示。(数组：0-FR，1-FL，2-RR, 3-RL)
    short foot_force_est[4];        //沿用的，但是目前不用。

    unsigned long tick;             //1ms 计时用，按照 1ms 递增。
    octet wireless_remote[40];      //遥控器原始数据。

    //&0x80 -  电机                 超时标志            1-超时    0-正常
    //&0x40 -  小 Mcu               超时标志            1-超时    0-正常
    //&0x20 -  遥控器               超时标志            1-超时    0-正常
    //&0x10 -  电池                 超时标志            1-超时    0-正常

    //&0x04 -  自动充电             自动充电状态标志    1-不充电
```

0-充电

//&0x02 - 板载电流错误标志 错误标志 1-板载电流异常

0-正常

//&0x01 - 运控命令超时 超时标志 1-超时

0-正常

octet bit_flag; //各个组件状态显示

float adc_reel; //卷线器电流（范围：0 - 3A)。

octet temperature_ntc1; //主板中心温度值（范围：-20 - 100℃)。

octet temperature_ntc2; //自动充电温度（范围：-20 - 100℃)。

float power_v; //此电压值为主板电压 -> 电池电压 。

float power_a; //此电流值为主板电流值 -> 电机电流。

unsigned short fan_frequency[4]; //风扇转速（目前可按照实际数值显示 0-10000)。(0-左后转速 ，1-右后转速，2-前转速，单位转/分钟）(堵转检测：3-&0x01：左后堵转 ，&0x02：右后堵转，&0x04：前堵转)

```
    unsigned long reserve;     //保留位。
    unsigned long crc;         //数据 CRC 校验用。


    };



    };   // module dds_

    };   // module msg

    };   // module unitree_go


    #endif   // __unitree_go__msg__low_state__idl__
```

电机状态信息：共 12 个电机，获取顺序可以按照如下进行，详情见以下的 MotorState_.idl 。

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
// with input from unitree_go:msg/MotorState.idl
// generated code does not contain a copyright notice


#ifndef __unitree_go__msg__motor_state__idl__
#define __unitree_go__msg__motor_state__idl__




module unitree_go {


module msg {


module dds_ {




struct MotorState_ {
octet  mode;        //电机控制模式（Foc 模式（工作模式）-> 0x01 ，
stop 模式（待机模式）-> 0x00。）
```

float q;　　//关机反馈位置信息：默认为弧度值（可按照实际情况改为角度值），可按照实际数值显示（弧度值范围：-7 - +7，显示 3 位小数）。

float dq;　　//关节反馈速度

float ddq;　　//关节反馈加速度

float tau_est;　//关节反馈力矩

float q_raw;　　//沿用的，但是目前不用。

float dq_raw;　　//沿用的，但是目前不用。

float ddq_raw;　//沿用的，但是目前不用。

octet temperature;　　//电机温度信息：类型：int8_t ，可按照实际数值显示（范围：-100 - 150）。

unsigned long lost;　　//电机丢包信息：可按照实际数值显示（范围：0-9999999999）。

unsigned long reserve[2];　　//当前电机通信频率+电机错误标志位：（数组：0-电机错误标志位（范围：0-255，可按照实际数值显示），1-当前电机通信频率（范围：0-800，可按照实际数值显示））

```
};



};  // module dds_

};  // module msg

};  // module unitree_go



#endif  // __unitree_go__msg__motor_state__idl__
```

IMU 状态信息：包含了三轴的加速度和角速度信息，四元数信息，欧拉角信息，温度信息，详情见以下的 IMUState_.idl。

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em

// with input from unitree_go:msg/MotorState.idl

// generated code does not contain a copyright notice
```

```
#ifndef __unitree_go__msg__motor_state__idl__

#define __unitree_go__msg__motor_state__idl__




module unitree_go {


module msg {


module dds_ {




struct MotorState_ {

octet  mode;        //电机控制模式（Foc 模式（工作模式）-> 0x01 ，
stop 模式（待机模式）-> 0x00。）

float q;            //关机反馈位置信息：默认为弧度值（可按照实际情况改
为角度值），可按照实际数值显示（弧度值范围：-7 - +7，显示 3 位小数）。

float dq;           //关节反馈速度

float ddq;          //关节反馈加速度
```

```
    float tau_est;  //关节反馈力矩


    float q_raw;     //沿用的，但是目前不用。
    float dq_raw;    //沿用的，但是目前不用。
    float ddq_raw;   //沿用的，但是目前不用。
    octet temperature;              //电机温度信息：类型：int8_t ，可按照实际数值显示（范围：-100 - 150）。
    unsigned long lost;             //电机丢包信息：可按照实际数值显示（范围：0-9999999999）。
    unsigned long reserve[2];    //当前电机通信频率+电机错误标志位：（数组：0-电机错误标志位（范围：0-255，可按照实际数值显示），1-当前电机通信频率（范围：0-800，可按照实际数值显示））


    };


};  // module dds_
```

```
};  // module msg


};  // module unitree_go



#endif  // __unitree_go__msg__motor_state__idl__
```

电池状态信息：包含了电池版本、状态信息、电池电量信息、充放电、循环次数、温度、单节电池电压。详情见以下的 BmsState_.idl。

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
// with input from unitree_go:msg/BmsState.idl
// generated code does not contain a copyright notice


#ifndef __unitree_go__msg__bms_state__idl__
#define __unitree_go__msg__bms_state__idl__
```

```
module unitree_go {


module msg {


module dds_ {




struct BmsState_ {
octet version_high;      //电池版本
octet version_low;       //电池版本

// 0：SAFE,（未开启电池）
// 1：WAKE_UP,（唤醒事件）

// 6：PRECHG, （电池预冲电中）
// 7：CHG, （电池正常充电中）
// 8：DCHG, （电池正常放电中）
// 9：SELF_DCHG, （电池自放电中）
```

```
// 11：ALARM, （电池存在警告）
// 12：RESET_ALARM, （等待按键复位警告中）
// 13：AUTO_RECOVERY （复位中）
octet status;                //电池状态信息。

octet soc;                   //电池电量信息：（类型：uint8_t）(范围 1% - 100%)
long current;                //充放电信息：(正：代表充电，负代表放电）可按照实际数值显示
unsigned short cycle;  //充电循环次数
octet bq_ntc[2];             //电池内部两个 NTC 的温度（int8_t）（范围：-100 - 150）。  0- BAT1; 1- BAT2

octet mcu_ntc[2];            //电池 NTC 数组：0 - RES，1 - MOS （int8_t）（范围：-100 - 150）。

unsigned short cell_vol[15];          //电池内部 15 节电池的电压。
```

```
};



};  // module dds_

};  // module msg

};  // module unitree_go



#endif  // __unitree_go__msg__bms_state__idl__
```

其他信息：

foot_force[4]：代表每条腿的足端力信息。顺序（0-FR，1-FL，2-RR, 3-RL）。

bit_flag：各个组件的状态信息，用于实时检测各个组件的状态，（1.检测电机、小 MCU、遥控器、电池、运控命令是否超时。2.检测

板载电流是否有错误标志，触发硬件电流保护就会产生错误标志。3.检测是否处于自动充电状态)。

fan_frequency[4]：风扇转速和堵转检测（0-左后转速，1-右后转速，2-前转速，单位转/分钟）（堵转检测：3-&0x01：左后堵转，&0x02：右后堵转，&0x04：前堵转)。

保护机制：adc_reel->卷线器电流，temperature_ntc1->主板中心温度值，temperature_ntc2->自动充电温度，power_v->电池电压，power_a->电机电流。

2、 底层运动控制例程

以下例程基于 C++语言，示范如何调用底层接口控制四足机器人的右前腿摆动。

2.1、 例程完整代码

```
import time
import sys
from unitree_sdk2py.core.channel import ChannelPublisher, ChannelFactoryInitialize
from unitree_sdk2py.core.channel import ChannelSubscriber, ChannelFactoryInitialize
```

```
    from unitree_sdk2py.idl.default import
unitree_go_msg_dds__LowCmd_
    from unitree_sdk2py.idl.unitree_go.msg.dds_ import LowCmd_
    from unitree_sdk2py.utils.crc import CRC
    from unitree_sdk2py.utils.thread import Thread
    import unitree_legged_const as go2
    crc = CRC()
    if __name__ == '__main__':
        if len(sys.argv)>1:
            ChannelFactoryInitialize(0, sys.argv[1])
        else:
            ChannelFactoryInitialize(0)
        # Create a publisher to publish the data defined in UserData
class
        pub = ChannelPublisher("rt/lowcmd", LowCmd_)
        pub.Init()
        cmd = unitree_go_msg_dds__LowCmd_()
        cmd.head[0]=0xFE
```

```
        cmd.head[1]=0xEF
        cmd.level_flag = 0xFF
        cmd.gpio = 0
        for i in range(20):
            cmd.motor_cmd[i].mode = 0x01   # (PMSM) mode
            cmd.motor_cmd[i].q= go2.PosStopF
            cmd.motor_cmd[i].kp = 0
            cmd.motor_cmd[i].dq = go2.VelStopF
            cmd.motor_cmd[i].kd = 0
            cmd.motor_cmd[i].tau = 0
        while True:
            # Toque controle, set RL_2 toque
            cmd.motor_cmd[go2.LegID["RL_2"]].q = 0.0 # Set to stop
position(rad)
            cmd.motor_cmd[go2.LegID["RL_2"]].kp = 0.0
            cmd.motor_cmd[go2.LegID["RL_2"]].dq = 0.0 # Set to stop
angular velocity(rad/s)
            cmd.motor_cmd[go2.LegID["RL_2"]].kd = 0.0
```

```
            cmd.motor_cmd[go2.LegID["RL_2"]].tau  =  1.0  #  target
toque is set to 1N.m
            # Poinstion(rad) control, set RL_0 rad
            cmd.motor_cmd[go2.LegID["RL_0"]].q  =  0.0    #  Taregt
angular(rad)
            cmd.motor_cmd[go2.LegID["RL_0"]].kp    =    10.0    #
Poinstion(rad) control kp gain
            cmd.motor_cmd[go2.LegID["RL_0"]].dq  =  0.0    #  Taregt
angular velocity(rad/ss)
            cmd.motor_cmd[go2.LegID["RL_0"]].kd    =    1.0       #
Poinstion(rad) control kd gain
            cmd.motor_cmd[go2.LegID["RL_0"]].tau    =    0.0    #
Feedforward toque 1N.m
            cmd.crc = crc.Crc(cmd)
            #Publish message
            if pub.Write(cmd):
                print("Publish success. msg:", cmd.crc)
            else:
```

```
            print("Waitting for subscriber.")

        time.sleep(0.002)
```

2.2、 程序解析

2.2.1、Custom 类

在 Custom 类中了定义与 DDS 通信相关的类与函数，还有一些与控制腿摆动的变量。

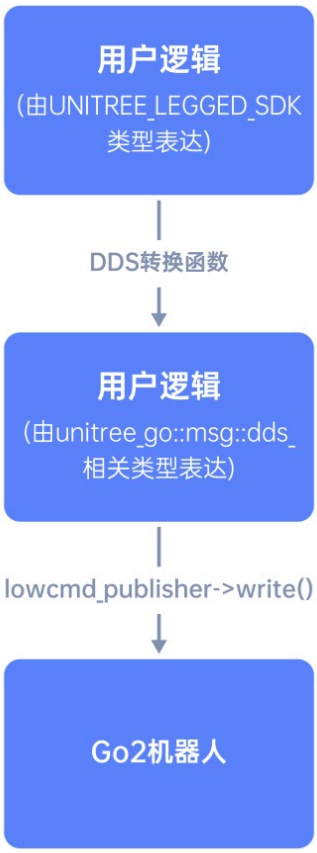
2.2.1.1、 成员变量（与腿部控制相关的变量）

名称	类型	说明
qInit	float类型数组	本例子中用于记录右前腿各关节的初始关节角度
qDes	float类型数组	本例子中用于表达右前腿各关节的期望关节角度
sin_mid_q	float类型数组	本例子中用于指定右前腿各关节周期往复运动中的中间关节角度
Kp	float类型数组	本例子中用于指定右前腿各关节的关节空间控制kp值
Kd	float类型数组	本例子中用于指定右前腿各关节的关节空间控制Kd值

2.2.1.2、 与通信相关的变量

名称	类型	说明
lowcmd_publisher	ChannelPublisherPtr<unitree_go::msg::dds_::LowCmd_>	底层命令发送对象,调用该对象的Write函数会向Go2机器人发送一次底层控制命令
lowstate_subscriber	ChannelSubscriberPtr<unitree_go::msg::dds_::LowState_>	底层状态信息订阅对象，底层状态包含电机信息、IMU数据、宇树手柄信息等
lowCmdWriteThreadPtr	ThreadPtr	DDS相关的底层命令发送线程指针
cmd	unitree_go::msg::dds_::LowCmd_	DDS相关的底层命令结构体，是最终通过DDS发送给Go2机器人，区别于LowCmd low_cmd
state	unitree_go::msg::dds_::LowState_	DDS相关的底层状态结构体
low_cmd	UNITREE_LEGGED_SDK::LowCmd	UNITREE_LEGGED_SDK类型的关于底层控制命令的结构体
low_state	UNITREE_LEGGED_SDK::low_state	UNITREE_LEGGED_SDK类型的关于底层状态信息的结构体

与通信相关的变量主要可以分为两种类型：UNITREE_LEGGED_SDK 类型 和 unitree_go::msg::dds_ 类型，它们的关系见于下图:



在本例程中，关于腿部摆动的控制用户逻辑首先用 UNITREE_LEGGED_SDK 类型变量进行表达，然后通过一些例程中提供的函数将其转换为 DDS 相关的类型，然后才将命令发至 Go2 机器人。这是因为 A1、Go1 等机器人的 SDK 也采用了 UNITREE_LEGGED_SDK 类型，此举为方便之前的用户延续与移植。

2.2.1.3、 **成员函数** InitLowCmd()

```
void Custom::InitLowCmd()
    {
        low_cmd.head[0] = 0xFE;
        low_cmd.head[1] = 0xEF;
        low_cmd.levelFlag = 0xFF;
        low_cmd.gpio = 0;

        for(int i=0; i<20; i++){
            low_cmd.motorCmd[i].mode = (0x01);   // motor switch to servo (PMSM) mode
            low_cmd.motorCmd[i].q = (PosStopF);
```

```
            low_cmd.motorCmd[i].Kp = (0);
            low_cmd.motorCmd[i].dq = (VelStopF);
            low_cmd.motorCmd[i].Kd = (0);
            low_cmd.motorCmd[i].tau = (0);
        }
    }
```

InitLowCmd() 该函数用于初始化设置 LowCmd 类型的 low_cmd 结构体。该函数放置于 Custom 类的构造函数运行一次即可。下面为其中一些要点：

LowCmd 类型中的 head 成员表示帧头，此帧头将用于 CRC 校验。head 、levelFlag、gpio 等按例程所示设置为默认值即可。

LowCmd 类型中有 20 个 motorCmd 成员，每一个成员的命令用于控制 Go2 机器人上相对应的一个电机，但 Go2 机器人上只有 12 个电机，故仅有前 12 个有效，剩余的 8 个起保留作用。

low_cmd.motorCmd[i].mode = (0x01); 此行命令中将 motorCmd 成员的 mode 变量设置为 0x01，0x01 表示将电机设置为伺服模式。如果用户在调试过程中发现无法控制 Go2 机器人

的关节电机，请检查变量的值是否为 0x01。

2.2.1.4、 LowStateMessageHandler

此函数的作用为关于底层状态信息接收的 DDS 回调函数，DDS 机制会以一定频率触发该回调函数。每次触发该回调函数都将更新一次底层状态信息。

2.2.1.5、 LowCmdwriteHandler()

此函数的作用为发送底层控制命令的回调函数，DDS 会以一定频率触发该回调函数。此回调函数的前大部分代码表示计算右前小腿摆动控制命令的用户逻辑，后小部分代码将控制命令发送至 Go2 机器人。

```
void LowCmdwriteHandler()
{
        motiontime++;

            for(int i=0; i<12; i++){
               printf("%f ", state.motor_state()[i].q());
            }
            printf("\n");
```

```
            // if( motiontime >= 100){
            if (motiontime >= 0){
                // first, get record initial position
                if (motiontime >= 0 && motiontime < 20){
                    qInit[0] = state.motor_state()[FR_0].q();
                    qInit[1] = state.motor_state()[FR_1].q();
                    qInit[2] = state.motor_state()[FR_2].q();
                }
                // second, move to the origin point of a sine
movement with Kp Kd
                if (motiontime >= 10 && motiontime < 400) {
                    rate_count++;
                    double rate = rate_count / 200.0; // needs count
to 200
                    Kp[0] = 5.0; Kp[1] = 5.0; Kp[2] = 5.0;
                    Kd[0] = 1.0; Kd[1] = 1.0; Kd[2] = 1.0;
```

```
                    qDes[0]     =     jointLinearInterpolation(qInit[0],
sin_mid_q[0], rate);
                    qDes[1]     =     jointLinearInterpolation(qInit[1],
sin_mid_q[1], rate);
                    qDes[2]     =     jointLinearInterpolation(qInit[2],
sin_mid_q[2], rate);
                }
                double sin_joint1, sin_joint2;
                // last, do sine wave
                float freq_Hz = 1;
                // float freq_Hz = 5;
                float freq_rad = freq_Hz * 2 * M_PI;
                float t = dt * sin_count;
                if (motiontime >= 400){
                    sin_count++;
                    sin_joint1 = 0.6 * sin(t * freq_rad);
                    sin_joint2 = -0.9 * sin(t * freq_rad);
                    qDes[0] = sin_mid_q[0];
```

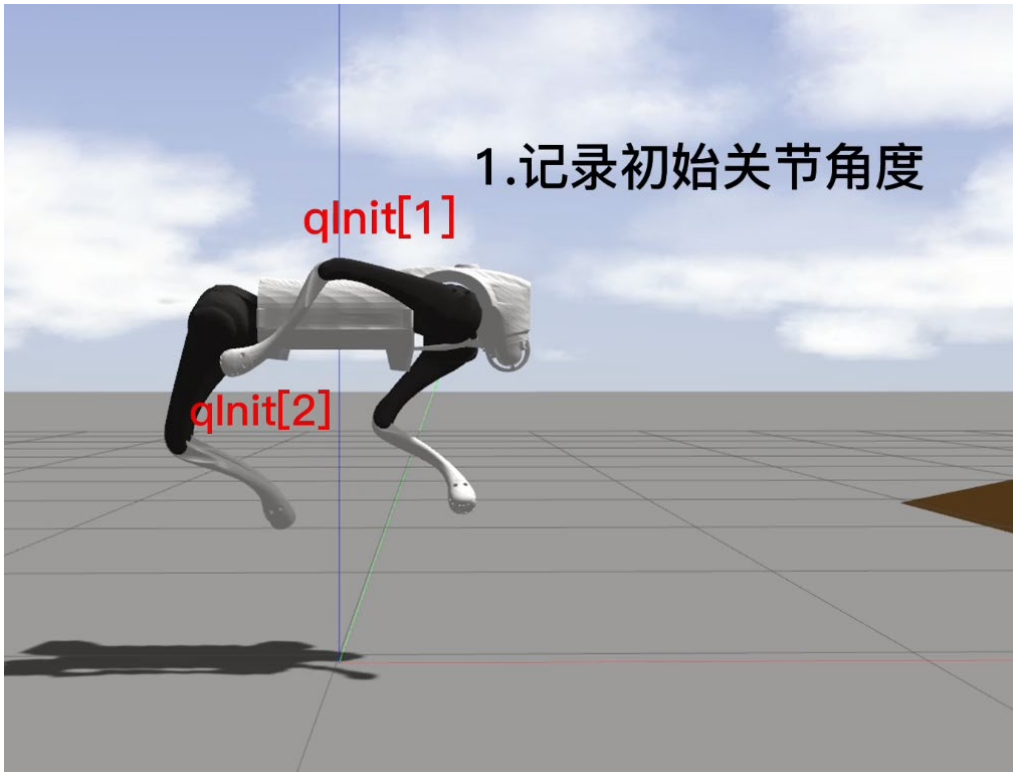
```
                    qDes[1] = sin_mid_q[1] + sin_joint1;
                    qDes[2] = sin_mid_q[2] + sin_joint2;
                }

                low_cmd.motorCmd[FR_2].q = qDes[2];
                low_cmd.motorCmd[FR_2].dq = 0;
                low_cmd.motorCmd[FR_2].Kp = Kp[2];
                low_cmd.motorCmd[FR_2].Kd = Kd[2];
                low_cmd.motorCmd[FR_2].tau = 0;
            }
            low_cmd.crc = crc32_core((uint32_t *)&low_cmd,
(sizeof(LowCmd)>>2)-1);

            lowCmd2Dds(low_cmd, cmd);
            lowcmd_publisher->Write(cmd);
        }
```

上述代码块中的三个 if 语句可将用户逻辑分为三部分：

①if (motiontime >= 0 && motiontime < 20) -> 记录初始关节角度

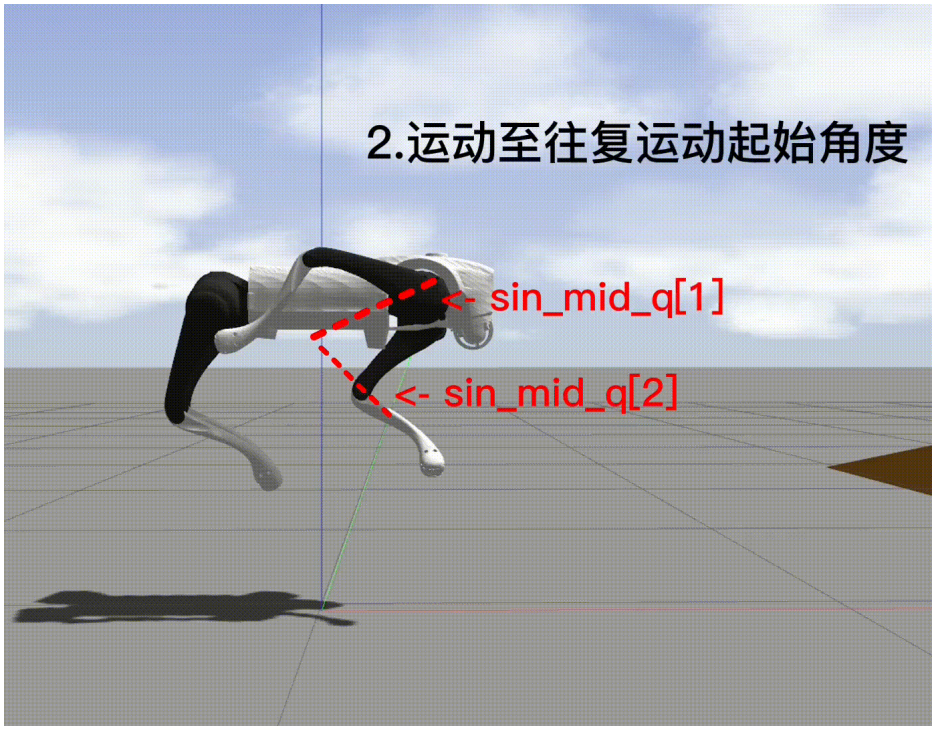



```
if (motiontime >= 0 && motiontime < 20)
{
    qInit[0] = state.motor_state()[FR_0].q();
    qInit[1] = state.motor_state()[FR_1].q();
    qInit[2] = state.motor_state()[FR_2].q();
}
```

该段代码表示当 motiontime 小于 20 时，会读取右前腿(FR)

的机身关节(FR_0)、大腿关节(FR_1)、小腿关节(FR_2)的关节角度值，并分别赋值给 qInit 数组中的元素。可以总结此段代码的作用是记录右前腿的初始位置，如上图所示。

②if (motiontime >= 10 && motiontime < 400) -> 运动至往复运动起始角度

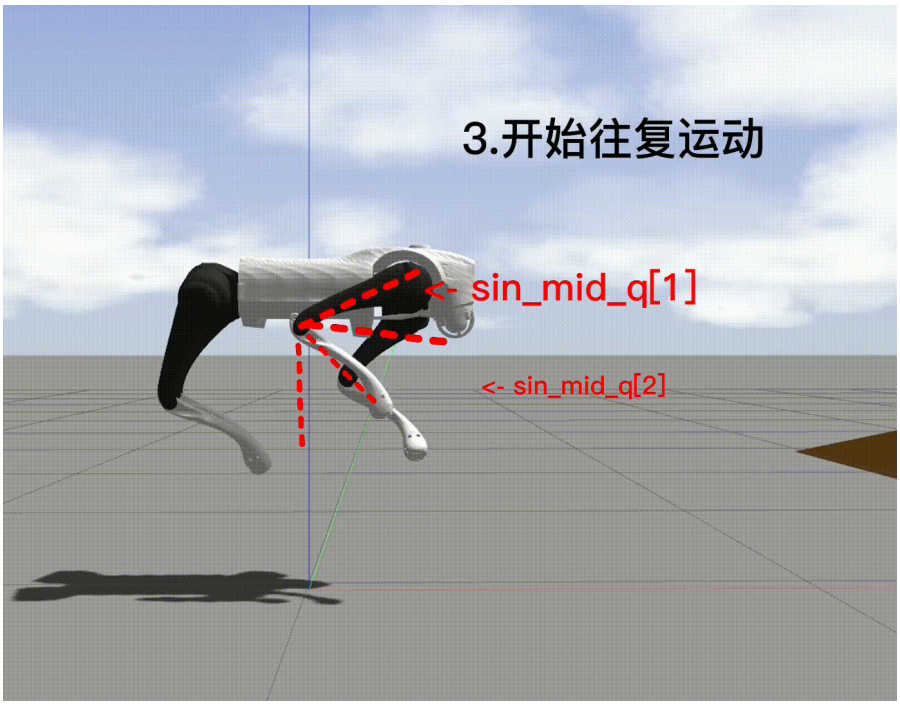



```
if (motiontime >= 10 && motiontime < 400) {
    rate_count++;
    double rate = rate_count / 200.0; // needs count to 200
    Kp[0] = 5.0; Kp[1] = 5.0; Kp[2] = 5.0;
    Kd[0] = 1.0; Kd[1] = 1.0; Kd[2] = 1.0;
```

```
        qDes[0] = jointLinearInterpolation(qInit[0], sin_mid_q[0], rate);
        qDes[1] = jointLinearInterpolation(qInit[1], sin_mid_q[1], rate);
        qDes[2] = jointLinearInterpolation(qInit[2], sin_mid_q[2], rate);
    }
}
```

此段代码的作用是计算控制命令 qDes， 该控制命令会驱动右前腿各关节缓慢地从 qInit 数组指定的角度 运动到 sim_mid_q 数组中指定的关节角度，如上图所示。

③if (motiontime >= 400) -> 开始往复运动



```
double sin_joint1, sin_joint2;

// last, do sine wave

float freq_Hz = 1;

// float freq_Hz = 5;

float freq_rad = freq_Hz * 2 * M_PI;

float t = dt * sin_count;

if (motiontime >= 400)

{

   sin_count++;

   sin_joint1 = 0.6 * sin(t * freq_rad);

   sin_joint2 = -0.9 * sin(t * freq_rad);

   qDes[0] = sin_mid_q[0];

   qDes[1] = sin_mid_q[1] + sin_joint1;

   qDes[2] = sin_mid_q[2] + sin_joint2;

}



low_cmd.motorCmd[FR_2].q = qDes[2];

low_cmd.motorCmd[FR_2].dq = 0;
```

```
    low_cmd.motorCmd[FR_2].Kp = Kp[2];
    low_cmd.motorCmd[FR_2].Kd = Kd[2];
    low_cmd.motorCmd[FR_2].tau = 0;
```

此段代码的作用是计算控制命令 qDes， 该控制命令会驱动右前腿中的小腿关节在 sim_mid_q 数组中指定的关节角度附近摆动。如上图所示，右前小腿将会在 sin_mid_q[2] ± 0.9 rad 范围内摆动。

```
    low_cmd.crc = crc32_core((uint32_t *)&low_cmd, (sizeof(LowCmd)>>2)-1);
    lowCmd2Dds(low_cmd, cmd);
    lowcmd_publisher->Write(cmd);
```

此段代码中第一行首先计算了 CRC 校验码。第二行代码通过 lowCmd2Dds() 函数将 UNITREE_LEGGED_SDK::low_cmd 类型的 low_cmd 变量 转换至 与 DDS 相关的 unitree_go::msg::dds_::LowCmd_ 类型的 cmd 变量。最后一行代码表示调用 lowcmd_publisher 的 Write() 函数将 控制命令 发送给 Go2 机器人。

2.2.2、 DDS 转换函数

以下是一系列与 DDS 相关的转换函数。当用户表达控制逻辑时，

UNITREE_LEGGED_SDK::LowCmd 和 UNITREE_LEGGED_SDK::LowState 是需要操作的两种类型的变量，因为这两种类型可读性更高。而实际给 Go2 机器人发命令或获取状态信息时，则通过 unitree_go::msg::dds_::LowCmd_ 和 unitree_go::msg::dds_::LowState_ 这两种类型的结构体。

故需要通过一些转换函数将这两种类型的结构体进行转换。用户重点关注 lowCmd2Dds() 和 dds2LowState() 转换函数即可。而对于中间函数，则可不必过分关注实现细节，使用时直接复制粘贴即可。

名称	说明
void dds2Imu(unitree_go::msg::dds_::IMUState_ & dds, IMU& raw)	关于 IMU 状态信息的转换函数，将unitree_go::msg::dds_::IMUState_转换至 UNITREE_LEGGED_SDK::IMU，为中间函数。
void dds2BmsState(unitree_go::msg::dds_::BmsState_& dds, BmsState& raw)	关于电池状态信息的转换函数，将unitree_go::msg::dds_::BmsState_转换至 UNITREE_LEGGED_SDK::BmsState，为中间函数。
void dds2MotorState(std::array<unitree_go::msg::dds_::MotorState_, 20>& dds, MotorState* raw)	关于电机状态信息的转换函数，将unitree_go::msg::dds_::MotorState_转换至 UNITREE_LEGGED_SDK::MotorState，为中间函数。
void dds2LowState(unitree_go::msg::dds_::LowState_& dds, LowState& raw)	将UNITREE_LEGGED_SDK::LowState 转换至unitree_go::msg::dds_::LowState_。
void motorCmd2Dds(MotorCmd* raw, std::array<unitree_go::msg::dds_::MotorCmd_, 20>& dds)	关于电机控制命令的转换函数，将UNITREE_LEGGED_SDK::MotorCmd转换至unitree_go::msg::dds_::MotorState_，为中间函数。
void bmsCmd2Dds(BmsCmd& raw, unitree_go::msg::dds_::BmsCmd_& dds)	关于电池管理命令的转换函数，将UNITREE_LEGGED_SDK::BmsCmd转换至unitree_go::msg::dds_::BmsCmd_ ，为中间函数。
void lowCmd2Dds(LowCmd& raw, unitree_go::msg::dds_::LowCmd_& dds)	将UNITREE_LEGGED_SDK::LowCmd 转换至unitree_go::msg::dds_::LowCmd_。
void crc32_core(uint32_t* ptr, uint32_t len)	crc校验函数，为中间函数。

2.3、 例程测试

2.3.1、 关闭运控服务

在运行该例程前需要先关闭 Go2 的主运控服务(sport_mode)。 可在 App-设置-服务状态 里点击对应的服务关闭。也可以通过调用服务状态开关接口关闭运控服务，关于此方式可关注 unitree_sdk2 中的 stand_example_go2 例程，在此例程中展示了如何通过调用 ServiceSwitch 服务接口关闭运控服务。



2.3.2、 运行例程

打开一个终端，并依次执行下列命令以运行例程。注：第二行命令中的"enpos3"字符串需要用户自行替换为用户电脑中对应于 123 网段的网卡名字。

```
cd ~/unitree_sdk2/build/bin

sudo ./low_level enp0s3
```

2.3.1、 运行结果确认

此时机器人右前小腿会呈规律性摆动状态。



3、 项目总结

在本项目的学习过程中，我们不仅掌握了 Go2 机器人的底层控制原理，还深入理解了其腿部的机械特性以及底层运动控制算法。这将使我们能够更有效地优化机器人的运动性能，实现更复杂和精确的动作控制。